

Nama : Aulia Tauhid Akbar
NIM : 250401020136
Jurusan : Informatika
Mata Kuliah : Struktur Data dan Algoritma

- **SIMULASI MANUAL**

Penyelesaian Hashing ($N = 11$, $H(k) = k \bmod 11$)

Langkah 0: Hitung nilai hash tiap key

Key	Perhitungan	H(k)
43	$43 \bmod 11$	10
22	$22 \bmod 11$	0
1	$1 \bmod 11$	1
12	$12 \bmod 11$	1
34	$34 \bmod 11$	1
56	$56 \bmod 11$	1
77	$77 \bmod 11$	0
88	$88 \bmod 11$	0

Terlihat bahwa banyak key akan bertabrakan di indeks **0** (22, 77, 88) dan indeks **1** (1, 12, 34, 56).

- Linear Probing

Rumus: $H(k, i) = (H(k) + i) \bmod 11$, $i = 0, 1, 2, \dots$

Langkah insert satu per satu:

- 1) $43 \rightarrow H=10 \rightarrow$ slot 10 kosong $\rightarrow$ taruh di 10
- 2) $22 \rightarrow H=0 \rightarrow$ slot 0 kosong $\rightarrow$ taruh di 0
- 3) $1 \rightarrow H=1 \rightarrow$ slot 1 kosong $\rightarrow$ taruh di 1
- 4) $12 \rightarrow H=1 \rightarrow$ slot 1 terisi (1) $\rightarrow i=1: (1+1)=2 \rightarrow$ slot 2 kosong $\rightarrow$ taruh di 2
- 5) $34 \rightarrow H=1 \rightarrow$ slot 1 terisi, $i=1$: slot 2 terisi, $i=2: (1+2)=3 \rightarrow$ slot 3 kosong $\rightarrow$ taruh di 3
- 6) $56 \rightarrow H=1 \rightarrow$ slot 1,2,3 terisi, $i=3: (1+3)=4 \rightarrow$ slot 4 kosong $\rightarrow$ taruh di 4
- 7) $77 \rightarrow H=0 \rightarrow$ slot 0 terisi (22), $i=1$: slot 1 terisi, $i=2$: slot 2 terisi, $i=3$: slot 3 terisi, $i=4$: slot 4 terisi, $i=5: (0+5)=5 \rightarrow$ slot 5 kosong $\rightarrow$ taruh di 5
- 8) $88 \rightarrow H=0 \rightarrow$ slot 0,1,2,3,4,5 semua terisi (cek berurutan $i=1..5$), $i=6: (0+6)=6 \rightarrow$ slot 6 kosong $\rightarrow$ taruh di 6

Hasil Akhir Linear Probing

Indeks	0	1	2	3	4	5	6	7	8	9	10
Isi	22	1	12	34	56	77	88	-	-	-	43

- Separate Chaining

Setiap indeks punya linked list. Key baru ditambahkan di akhir list pada indeks hasil $H(k)$, tanpa perlu mencari slot lain.

Langkah insert:

- 1) $43 \rightarrow H=10 \rightarrow \text{list}[10]: 43$
- 2) $22 \rightarrow H=0 \rightarrow \text{list}[0]: 22$
- 3) $1 \rightarrow H=1 \rightarrow \text{list}[1]: 1$
- 4) $12 \rightarrow H=1 \rightarrow \text{tabrakan, tambah di akhir} \rightarrow \text{list}[1]: 1 \rightarrow 12$
- 5) $34 \rightarrow H=1 \rightarrow \text{tambah di akhir} \rightarrow \text{list}[1]: 1 \rightarrow 12 \rightarrow 34$
- 6) $56 \rightarrow H=1 \rightarrow \text{tambah di akhir} \rightarrow \text{list}[1]: 1 \rightarrow 12 \rightarrow 34 \rightarrow 56$
- 7) $77 \rightarrow H=0 \rightarrow \text{tabrakan, tambah di akhir} \rightarrow \text{list}[0]: 22 \rightarrow 77$
- 8) $88 \rightarrow H=0 \rightarrow \text{tambah di akhir} \rightarrow \text{list}[0]: 22 \rightarrow 77 \rightarrow 88$

Hasil Akhir Separate Chaining

Indeks	Isi (Linked List)
0	$22 \rightarrow 77 \rightarrow 88$
1	$1 \rightarrow 12 \rightarrow 34 \rightarrow 56$
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL
9	NULL
10	43

Ringkasan Perbedaan

- Linear Probing: semua key tersebar ke slot kosong terdekat (indeks 0–6 dan 10 terisi), sehingga load factor tabel tampak penuh tapi setiap slot hanya menyimpan 1 elemen.
- Separate Chaining: elemen yang bertabrakan tetap berada di indeks hash aslinya (0 dan 1), tetapi disimpan sebagai list berantai, sehingga sebagian besar slot lain (2–9) tetap kosong.

• IMPLEMENTASI PEMROGRAMAN

HashTable.java

```
import java.util.Scanner;

public class HashTable {

    // ----- Node untuk Linked List (chaining) -----
    private static class Node {
        int key;
        String value;
        Node next;

        Node(int key, String value) {
            this.key = key;
            this.value = value;
            this.next = null;
        }
    }

    // ----- Atribut Hash Table -----
    private Node[] table; // array of linked list (chain)
    private int capacity; // ukuran array
    private int size; // jumlah elemen yang tersimpan

    // ----- Konstruktor -----
    public HashTable() {
        this.capacity = 10; // ukuran awal array = 10
        this.table = new Node[capacity];
        this.size = 0;
    }

    // ----- Fungsi Hash (metode pembagian / modulo) -----
    private int hashFunction(int key) {
        // Math.abs untuk berjaga-jaga jika key negatif
        return Math.abs(key) % capacity;
    }
}
```

```

}

// ----- a. insert(key, value) -----
public void insert(int key, String value) {
    int index = hashFunction(key);
    Node head = table[index];

    // Jika key sudah ada, update value-nya (tidak membuat duplikat)
    Node current = head;
    while (current != null) {
        if (current.key == key) {
            current.value = value;
            System.out.println("Key " + key + " sudah ada, value diperbarui menjadi \"" +
value + "\"" pada indeks " + index);
            return;
        }
        current = current.next;
    }

    // Buat node baru dan sisipkan di AKHIR linked list (chaining)
    Node newNode = new Node(key, value);

    if (head == null) {
        table[index] = newNode;
    } else {
        Node temp = head;
        while (temp.next != null) {
            temp = temp.next;
        }
        temp.next = newNode;
    }

    size++;
    System.out.println("Insert -> key: " + key + ", value: \"" + value + "\"" ke indeks " +
index);
}

// ----- b. search(key) -----
public String search(int key) {
    int index = hashFunction(key);
    Node current = table[index];

    while (current != null) {
        if (current.key == key) {
            return current.value;
        }
    }
}

```

```

    }
    current = current.next;
}
return "Tidak ditemukan";
}

// ----- c. remove(key) -----
public boolean remove(int key) {
    int index = hashFunction(key);
    Node current = table[index];
    Node previous = null;

    while (current != null) {
        if (current.key == key) {
            if (previous == null) {
                // Node yang dihapus adalah head dari list
                table[index] = current.next;
            } else {
                previous.next = current.next;
            }
            size--;
            System.out.println("Key " + key + " berhasil dihapus dari indeks " + index);
            return true;
        }
        previous = current;
        current = current.next;
    }

    System.out.println("Key " + key + " tidak ditemukan, tidak ada yang dihapus");
    return false;
}

// ----- d. display() -----
public void display() {
    System.out.println("\n===== Isi Hash Table (kapasitas = " + capacity + ", jumlah elemen " + size + ") =====");
    for (int i = 0; i < capacity; i++) {
        System.out.print("Indeks " + i + ": ");
        Node current = table[i];

        if (current == null) {
            System.out.println("kosong");
        } else {
            StringBuilder sb = new StringBuilder();
            while (current != null) {

```

```

        sb.append("[").append(current.key).append(" ->
").append(current.value).append("]");
        if (current.next != null) {
            sb.append(" -> ");
        }
        current = current.next;
    }
    System.out.println(sb.toString());
}
}

System.out.println("=====\n");
}

// ----- Getter tambahan (opsional, berguna untuk verifikasi test) -----
public int getSize() {
    return size;
}

public int getCapacity() {
    return capacity;
}

// ----- Program Utama: MENU INTERAKTIF -----
public static void main(String[] args) {
    HashTable ht = new HashTable();
    Scanner sc = new Scanner(System.in);
    boolean running = true;

    System.out.println("=====");
    System.out.println(" PROGRAM HASH TABLE (Separate Chaining) - MODE MANUAL ");
    System.out.println("=====");

    while (running) {
        System.out.println("\nPilih operasi yang ingin diuji:");
        System.out.println("1. Insert (key, value)");
        System.out.println("2. Search (key)");
        System.out.println("3. Remove (key)");
        System.out.println("4. Display seluruh Hash Table");
        System.out.println("5. Tampilkan jumlah elemen & kapasitas");
        System.out.println("0. Keluar");
        System.out.print("Masukkan pilihan: ");

        String pilihanStr = sc.nextLine().trim();
        int pilihan;
        try {

```

```

        pilihan = Integer.parseInt(pilihanStr);
    } catch (NumberFormatException e) {
        System.out.println("Input tidak valid, masukkan angka menu (0-5).");
        continue;
    }

    switch (pilihan) {
        case 1: {
            System.out.print("Masukkan key (angka bulat): ");
            int key = bacaInt(sc);
            System.out.print("Masukkan value (teks): ");
            String value = sc.nextLine();
            ht.insert(key, value);
            break;
        }
        case 2: {
            System.out.print("Masukkan key yang dicari: ");
            int key = bacaInt(sc);
            String hasil = ht.search(key);
            System.out.println("Hasil pencarian key " + key + " -> " + hasil);
            break;
        }
        case 3: {
            System.out.print("Masukkan key yang ingin dihapus: ");
            int key = bacaInt(sc);
            ht.remove(key);
            break;
        }
        case 4:
            ht.display();
            break;
        case 5:
            System.out.println("Jumlah elemen tersimpan : " + ht.getSize());
            System.out.println("Kapasitas array          : " + ht.getCapacity());
            break;
        case 0:
            running = false;
            System.out.println("Program selesai. Terima kasih!");
            break;
        default:
            System.out.println("Pilihan tidak dikenal, silakan pilih 0-5.");
    }
}

sc.close();

```

```

    }

    // Helper untuk membaca input integer dengan validasi sederhana
    private static int bacaInt(Scanner sc) {
        while (true) {
            String input = sc.nextLine().trim();
            try {
                return Integer.parseInt(input);
            } catch (NumberFormatException e) {
                System.out.print("Input harus berupa angka bulat, coba lagi: ");
            }
        }
    }
}
}

```

Test Case

- **Operasi Pengujian**

```

Pilih operasi yang ingin diuji:
1. Insert (key, value)
2. Search (key)
3. Remove (key)
4. Display seluruh Hash Table
5. Tampilkan jumlah elemen & kapasitas
0. Keluar
Masukkan pilihan: 4

===== Isi Hash Table (kapasitas = 10, jumlah elemen = 0) =====
Indeks 0: kosong
Indeks 1: kosong
Indeks 2: kosong
Indeks 3: kosong
Indeks 4: kosong
Indeks 5: kosong
Indeks 6: kosong
Indeks 7: kosong
Indeks 8: kosong
Indeks 9: kosong
=====

```


- **Insert Key**

```
Pilih operasi yang ingin diuji:
1. Insert (key, value)
2. Search (key)
3. Remove (key)
4. Display seluruh Hash Table
5. Tampilkan jumlah elemen & kapasitas
0. Keluar
Masukkan pilihan: 1
Masukkan key (angka bulat): 4
Masukkan value (teks): Empat
Insert -> key: 4, value: "Empat" ke indeks 4

Pilih operasi yang ingin diuji:
1. Insert (key, value)
2. Search (key)
3. Remove (key)
4. Display seluruh Hash Table
5. Tampilkan jumlah elemen & kapasitas
0. Keluar
Masukkan pilihan: 4

===== Isi Hash Table (kapasitas = 10, jumlah elemen = 1) =====
Indeks 0: kosong
Indeks 1: kosong
Indeks 2: kosong
Indeks 3: kosong
Indeks 4: [4 -> Empat]
Indeks 5: kosong
Indeks 6: kosong
Indeks 7: kosong
Indeks 8: kosong
Indeks 9: kosong
=====
```

- **Search Key**

```
Pilih operasi yang ingin diuji:
1. Insert (key, value)
2. Search (key)
3. Remove (key)
4. Display seluruh Hash Table
5. Tampilkan jumlah elemen & kapasitas
0. Keluar
Masukkan pilihan: 2
Masukkan key yang dicari: 4
Hasil pencarian key 4 -> Empat
```

- **Delete Key**

```
Masukkan pilihan: 3
Masukkan key yang ingin dihapus: 4
Key 4 berhasil dihapus dari indeks 4

Pilih operasi yang ingin diuji:
1. Insert (key, value)
2. Search (key)
3. Remove (key)
4. Display seluruh Hash Table
5. Tampilkan jumlah elemen & kapasitas
0. Keluar
Masukkan pilihan: 4

===== Isi Hash Table (kapasitas = 10, jumlah elemen = 0) =====
Indeks 0: kosong
Indeks 1: kosong
Indeks 2: kosong
Indeks 3: kosong
Indeks 4: kosong
Indeks 5: kosong
Indeks 6: kosong
Indeks 7: kosong
Indeks 8: kosong
Indeks 9: kosong
=====

Pilih operasi yang ingin diuji:
1. Insert (key, value)
2. Search (key)
3. Remove (key)
4. Display seluruh Hash Table
5. Tampilkan jumlah elemen & kapasitas
0. Keluar
Masukkan pilihan: █
```

- **Show Key**

```
Pilih operasi yang ingin diuji:
1. Insert (key, value)
2. Search (key)
3. Remove (key)
4. Display seluruh Hash Table
5. Tampilkan jumlah elemen & kapasitas
0. Keluar
Masukkan pilihan: 5
Jumlah elemen tersimpan : 0
Kapasitas array          : 10

Pilih operasi yang ingin diuji:
1. Insert (key, value)
2. Search (key)
3. Remove (key)
4. Display seluruh Hash Table
5. Tampilkan jumlah elemen & kapasitas
0. Keluar
```